

AIRPORT MANAGEMENT INFORMATION SYSTEM

Database Management Systems and Programming I – Term Project

Prepared By

- Selin Akan
- Melih Yurt
- Eren Siki

Instructor

Submission Date

24/05/2026

1. Introduction

The purpose of this project is to design and implement a relational database management system for an Airport Management Information System. The database was created for Ercan Airport Management in order to store, manipulate, and retrieve airport-related information efficiently.

The system manages airplanes, plane models, hangars, employees, technicians, traffic controllers, aircraft tests, and test events. The database structure was implemented using MySQL and MySQL Workbench.

The project demonstrates the use of:

- Relational database design
- Primary and foreign keys
- Entity relationships
- SQL queries
- Aggregate functions
- JOIN operations
- GROUP BY operations
- Data manipulation and management

The database simulates a real-life airport management environment and provides statistical information that can support airport administration and operational management.

2. Assumptions

The following assumptions were made while designing the database:

- Each airplane belongs to one plane model.
- One plane model can belong to multiple airplanes.
- Each employee has a unique social security number (SSN).
- Technicians are also employees.
- Traffic controllers are also employees.
- One technician can specialize in multiple plane models.
- One airplane can enter multiple hangars at different times.
- Hangar usage records contain entry and exit date/time information.
- Each test event belongs to one airplane, one technician, and one test.
- Test scores are evaluated over 100 points.
- One technician can perform multiple tests.

3. ER Diagram

The ER Diagram of the Airport Management Information System was created using MySQL Workbench.

(Insert ER Diagram Image Here)

4. Relational Data Model

Plane_Models

- model_id (PK)
- model_name
- manufacturer
- engine_type

Planes

- plane_id (PK)
- plane_no
- model_id (FK)
- capacity
- manufacture_year
- status

Hangars

- hangar_id (PK)

- hangar_name
- location
- capacity

Hangar_Usage

- usage_id (PK)
- plane_id (FK)
- hangar_id (FK)
- entry_date
- exit_date

Employees

- ssn (PK)
- first_name
- last_name
- phone
- salary
- union_membership_no

Technicians

- technician_id (PK)
- ssn (FK)
- specialization_level

Traffic_Controllers

- controller_id (PK)
- ssn (FK)
- controller_name
- last_medical_exam

Technician_Expertise

- technician_id (FK)
- model_id (FK)

Tests

- test_id (PK)
- test_name
- max_score
- duration_hours

Test_Events

- event_id (PK)

- plane_id (FK)
- technician_id (FK)
- test_id (FK)
- test_date
- hours_spent
- score

5. Data Definition Language (DDL)

Creating Plane_Models Table

```
CREATE TABLE Plane_Models (
    model_id INT PRIMARY KEY AUTO_INCREMENT,
    model_name VARCHAR(100) NOT NULL,
    manufacturer VARCHAR(100),
    engine_type VARCHAR(50)
);
```

Creating Planes Table

```
CREATE TABLE Planes (
    plane_id INT PRIMARY KEY AUTO_INCREMENT,
    plane_no VARCHAR(20) UNIQUE,
    model_id INT,
    capacity INT,
    manufacture_year YEAR,
    status VARCHAR(50),

    FOREIGN KEY (model_id)
    REFERENCES Plane_Models(model_id)
);
```

Creating Employees Table

```
CREATE TABLE Employees (
    ssn VARCHAR(20) PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    phone VARCHAR(20),
    salary DECIMAL(10,2),
    union_membership_no VARCHAR(50)
```

```
);
```

Creating Technicians Table

```
CREATE TABLE Technicians (  
    technician_id INT PRIMARY KEY AUTO_INCREMENT,  
    ssn VARCHAR(20),  
    specialization_level VARCHAR(50),  
  
    FOREIGN KEY (ssn)  
    REFERENCES Employees(ssn)  
);
```

Creating Tests Table

```
CREATE TABLE Tests (  
    test_id INT PRIMARY KEY AUTO_INCREMENT,  
    test_name VARCHAR(100),  
    max_score INT,  
    duration_hours INT  
);
```

Creating Test_Events Table

```
CREATE TABLE Test_Events (  
    event_id INT PRIMARY KEY AUTO_INCREMENT,  
    plane_id INT,  
    technician_id INT,  
    test_id INT,  
    test_date DATE,  
    hours_spent DECIMAL(5,2),  
    score INT,  
  
    FOREIGN KEY (plane_id)  
    REFERENCES Planes(plane_id),  
  
    FOREIGN KEY (technician_id)  
    REFERENCES Technicians(technician_id),  
  
    FOREIGN KEY (test_id)  
    REFERENCES Tests(test_id)  
);
```

6. Data Manipulation Language (DML)

Sample Data for Plane Models

```
INSERT INTO Plane_Models
(model_name, manufacturer, engine_type)
VALUES
('Boeing 737', 'AJet', 'Jet'),
('Airbus A320', 'Pegasus', 'Jet'),
('Boeing 777', 'Turkish Airlines', 'Jet');
```

Sample Data for Planes

```
INSERT INTO Planes
(plane_no, model_id, capacity, manufacture_year, status)
VALUES
('AJT1001', 1, 180, 2018, 'Active'),
('PGS2001', 2, 220, 2020, 'Maintenance'),
('THY3001', 3, 350, 2022, 'Active');
```

Sample Data for Employees

```
INSERT INTO Employees
(ssn, first_name, last_name, phone, salary, union_membership_no)
VALUES
('111111111', 'Ahmet', 'Yilmaz', '05551111111', 45000, 'UNION101'),
('222222222', 'Mehmet', 'Demir', '05552222222', 47000, 'UNION102');
```

7. SQL Queries

Query 1 – Display all airplanes

```
SELECT * FROM Planes;
```

Query 2 – Highest test score

```
SELECT MAX(score) AS highest_score
FROM Test_Events;
```

Query 3 – Average test score

```
SELECT AVG(score) AS average_score  
FROM Test_Events;
```

Query 4 – Number of tests performed by technicians

```
SELECT technician_id, COUNT(*) AS total_tests  
FROM Test_Events  
GROUP BY technician_id;
```

Query 5 – Airplane with highest capacity

```
SELECT *  
FROM Planes  
ORDER BY capacity DESC  
LIMIT 1;
```

Query 6 – Maintenance airplanes

```
SELECT *  
FROM Planes  
WHERE status = 'Maintenance';
```

Query 7 – Test scores above 80

```
SELECT *  
FROM Test_Events  
WHERE score > 80;
```

Query 8 – Average employee salary

```
SELECT AVG(salary) AS average_salary  
FROM Employees;
```

Query 9 – Highest paid employee

```
SELECT *  
FROM Employees  
ORDER BY salary DESC  
LIMIT 1;
```

Query 10 – Airplanes and their tests

```
SELECT
Planes.plane_no,
Tests.test_name,
Test_Events.score
FROM Test_Events
JOIN Planes
ON Test_Events.plane_id = Planes.plane_id
JOIN Tests
ON Test_Events.test_id = Tests.test_id;
```

Query 11 – Technician specialization levels

```
SELECT specialization_level, COUNT(*) AS total
FROM Technicians
GROUP BY specialization_level;
```

Query 12 – Tests performed in 2026

```
SELECT *
FROM Test_Events
WHERE YEAR(test_date) = 2026;
```

Query 13 – Hangars with capacity greater than 5

```
SELECT *
FROM Hangars
WHERE capacity > 5;
```

Query 14 – Total hours spent on tests

```
SELECT SUM(hours_spent) AS total_hours
FROM Test_Events;
```

Query 15 – Average test score for each airplane

```
SELECT plane_id, AVG(score) AS average_score
FROM Test_Events
GROUP BY plane_id
ORDER BY average_score DESC;
```


Query 16 – Airplane hangar usage information

```
SELECT
Planes.plane_no,
Hangars.hangar_name,
Hangar_Usage.entry_date,
Hangar_Usage.exit_date
FROM Hangar_Usage
JOIN Planes
ON Hangar_Usage.plane_id = Planes.plane_id
JOIN Hangars
ON Hangar_Usage.hangar_id = Hangars.hangar_id;
```

Query 17 – Technician expertise information

```
SELECT
Employees.first_name,
Employees.last_name,
Plane_Models.model_name
FROM Technician_Expertise
JOIN Technicians
ON Technician_Expertise.technician_id = Technicians.technician_id
JOIN Employees
ON Technicians.ssn = Employees.ssn
JOIN Plane_Models
ON Technician_Expertise.model_id = Plane_Models.model_id;
```

Query 18 – Traffic controllers with recent medical examination

```
SELECT *
FROM Traffic_Controllers
WHERE last_medical_exam > '2026-01-01';
```

8. Conclusion

In this project, a relational database management system for airport operations was successfully designed and implemented using MySQL.

The project includes multiple related tables, primary and foreign keys, SQL queries, and statistical reports. The system efficiently manages airplanes, employees, technicians, tests, hangars, and airport operations.

This project improved practical knowledge about:

- Relational database systems
- SQL programming
- Entity relationships
- Database normalization
- Database management principles

The Airport Management Information System provides an organized and realistic database structure suitable for airport administration and operational management.